

LEGACY SERIES — PHASE 1 INTERACTIVE BUILD SPEC

For: Stef

Base visual reference: Chassis Blueprint (greyed-out state, Image 2)

Coordinate tooling reference: Bento Coordinate Target Matrix (Image 1)

1. DEFAULT STATE = GREYED CHASSIS BLUEPRINT

The greyed-out chassis blueprint is not a placeholder — it is the canonical **default/reset layout**. Every module's default X/Y/W/H comes from this image, exactly as measured:

javascript

```
const DEFAULT_LAYOUT = {
  master: { x: 0, y: 0, width: 3840, height: 2160, locked: true },
  tiles: [
    { id: "power_amplifier", x: 120, y: 80, width: 3600, height: 380 },
    { id: "preamplifier", x: 120, y: 500, width: 3600, height: 320 },
    { id: "patchboard", x: 120, y: 860, width: 3600, height: 240 },
    { id: "dsp_matrix", x: 120, y: 1120, width: 3600, height: 220 },
    { id: "sound_modulators", x: 120, y: 1360, width: 3600, height: 180 },
    { id: "eq_10band", x: 120, y: 1560, width: 3600, height: 280 },
    { id: "output_stage", x: 120, y: 1860, width: 3600, height: 180 }
  ]
};
```

Reset must restore this object exactly — not a visual approximation of it. `(RESET)` = `setLayout(DEFAULT_LAYOUT)` + `(gsap.set())` all dials/switches/LEDs back to their default rotation/state (no animated transition on reset — snap instantly, this is a "return to known state" action, not a performance).

2. CONTROL ELEMENT REGISTRY (from Image 2 centerpoints)

These are the exact interactive elements and their pivot points. Every dial rotates around its `(cx, cy)`. Every switch/LED is a fixed-position toggle. Do not re-derive these — use as-is:

javascript

```

const CONTROLS = {
  dials: [
    { id: "volume",      cx: 720,  cy: 660, radius: 70, min: -60, max: 12, d
    { id: "balance",     cx: 1100, cy: 660, radius: 50, min: -100, max: 100,
    { id: "input_gain",  cx: 1420, cy: 660, radius: 50, min: -12, max: 12, d
    { id: "sub_level",   cx: 1740, cy: 660, radius: 50, min: -12, max: 12, d
    { id: "center_level", cx: 2060, cy: 660, radius: 50, min: -12, max: 12, d
    { id: "crossover_freq", cx: 3120, cy: 660, radius: 70, min: 40, max: 200,
  ],
  buttons: [
    { id: "power", cx: 210, cy: 240, radius: 36, type: "toggle" }
  ],
  meters: [
    { id: "vu_left",  cx: 1350, cy: 230, radius: 120 },
    { id: "vu_right", cx: 2490, cy: 230, radius: 120 }
  ],
  displays: [
    { id: "bitrate_display", cx: 2880, cy: 1230, width: 240, height: 60 }
  ],
  heatsinks: [
    { id: "left_heatsink",  cx: 500,  cy: 230, width: 280, height: 300 },
    { id: "right_heatsink", cx: 3340, cy: 230, width: 280, height: 300 }
  ],
  eq_bands: [
    "31Hz", "62Hz", "125Hz", "250Hz", "500Hz", "1kHz", "2kHz", "4kHz", "8kHz", "16kHz"
  ] // 10 vertical faders, evenly spaced across eq_10band tile, range -12..+12,
};

```

3. GSAP INTERACTION LAYER

Dials — drag-to-rotate (vertical drag or circular drag, your call, but pick one consistently across all dials):

javascript

```

gsap.to(dialRef.current, {
  rotation: mapValueToDegrees(value, min, max), // e.g. -135deg to +135deg sweep
  duration: 0.15,
  ease: "power2.out",
  transformOrigin: "center center"
});

```

Switches/toggles — snap with subtle overshoot, not linear:

javascript

```
gsap.to(switchRef.current, {  
  rotate: isOn ? 25 : -25, // or x-translate for slide switches  
  duration: 0.18,  
  ease: "back.out(1.7)"  
});
```

LEDs — glow pulse on state change, not a hard cut:

javascript

```
gsap.fromTo(ledRef.current,  
  { boxShadow: "0 0 0px currentColor" },  
  { boxShadow: "0 0 8px 2px currentColor", duration: 0.25, ease: "power1.out" }  
);
```

Mouse response should be subtle everywhere — no bouncy/cartoonish easing. Stick to `power2`/`power1`/short `back.out` with low overshoot. This is a premium hi-fi chassis, not a toy.

Hover — 1-2% scale + faint highlight ring on the active control only, never the whole tile:

javascript

```
gsap.to(el, { scale: 1.015, duration: 0.12, ease: "power1.out" });
```

4. REACT TRANSITION LAYER (tile-level: resize/reposition)

Tile drag/resize should feel physically damped, not instant-snap-then-settle. Use GSAP for the tile transform, driven by React state as the source of truth for X/Y/W/H (state updates on every frame during drag — no debounce, the overlay readout in Section 5 depends on this).

javascript

```

useLayoutEffect(() => {
  gsap.to(tileRef.current, {
    x: tile.x,
    y: tile.y,
    width: tile.width,
    height: tile.height,
    duration: isDragging ? 0 : 0.2, // zero duration while actively dragging, e
    ease: "power2.out"
  });
}, [tile.x, tile.y, tile.width, tile.height, isDragging]);

```

Grid snap (2px/3px/off) applies on drag-end, not during — so the tile follows the pointer exactly, then eases to the nearest grid value on release.

5. COORDINATE SYNC — WIRE IN THE EXISTING TOOL

Reuse the Bento Coordinate Target Matrix app you already built (Image 1) as the live readout/editor. Do not rebuild it — integrate it as the geometry panel for the selected tile.

Sync contract, both directions:

- **Drag/resize → readout:** on every pointermove during an active drag or resize, push `{ id, x, y, width, height }` into the same state object the coordinate app reads from. Update on every frame, not throttled — the panel in Image 1 already shows this live `(x 1122 / Y 783 / W 1530 / H 420)` style overlay).
- **Readout → tile:** typing a value into X/Y/W/H in the coordinate panel calls the same `setTile()` action the drag handler uses. No separate code path — one function, two entry points (pointer, keyboard).
- **Units:** integer pixels only, in the 3840×2160 master coordinate space — never viewport-scaled values. If the browser is rendering the canvas at 50% scale, the readout still shows true 4K coordinates (see viewport scaling rule already established in the Phase 1 base spec).
- **Grid snap indicator:** coordinate panel should reflect current snap mode (OFF / 2PX / 3PX) exactly as shown in Image 1's top bar.

6. RESET BUTTON

RESET FACTORY GRID

- Restores `DEFAULT_LAYOUT` (Section 1) instantly, no easing.

- Restores every dial/switch/LED to its `default` value from the CONTROLS registry — also instant, not animated (a reset should read as "state restored," not "state animated back").
 - Does **not** touch any saved custom layouts (Section 7) — reset only affects the currently active/working layout.
-

7. SAVE AS BUTTON

New requirement — add alongside the existing `COPY PRODUCTION ARRAY` / `RESET FACTORY GRID` buttons in the HUD bar:

SAVE AS...

- Opens a small named-save prompt (inline field, not a modal blocking the whole canvas — keep it light).
 - Persists the **current full state**: all tile X/Y/W/H + all dial/switch values, keyed by the name given.
 - Saved layouts populate a dropdown/list next to `RESET FACTORY GRID` so any saved layout can be recalled the same way factory defaults are recalled.
 - Storage: since this plugs into your existing `command_control` / Vistamations infra rather than a throwaway artifact, persist via your actual backend (SQLite/whatever Rachael or the relevant service already uses) — not browser localStorage, since this needs to survive across devices/sessions like the rest of your memory-bank data.
 - Loading a saved layout uses the exact same `setLayout()` call as Reset — just with the saved object instead of `DEFAULT_LAYOUT`.
-

8. BUILD ORDER FOR STEF

1. Render static chassis at `DEFAULT_LAYOUT` geometry — no interactivity yet, confirm pixel-exact match to Image 2.
2. Wire dial/switch/LED interactivity per Section 2-3 CONTROLS registry.
3. Wire tile drag/resize per Section 4, confirm state updates every frame.
4. Integrate coordinate panel per Section 5 — confirm bidirectional sync (drag moves numbers, typing numbers moves tile).
5. Add Reset (Section 6).
6. Add Save As + saved-layout recall (Section 7).
7. Confirm grid snap (2px/3px/off) applies correctly across drag, resize, and typed input.

Do not skip ahead to Phase 2 modules (recorder, mixer, guitar room) — this spec only covers Phase 1 chassis interactivity + the layout tooling around it.